Introduction to Programming with Python

2nd term 2025 – 2026, Bo HUANG

Programming basics

Python is a programming language that lets you work quickly and integrate systems more effectively

code or **source code**: The sequence of instructions in a program.

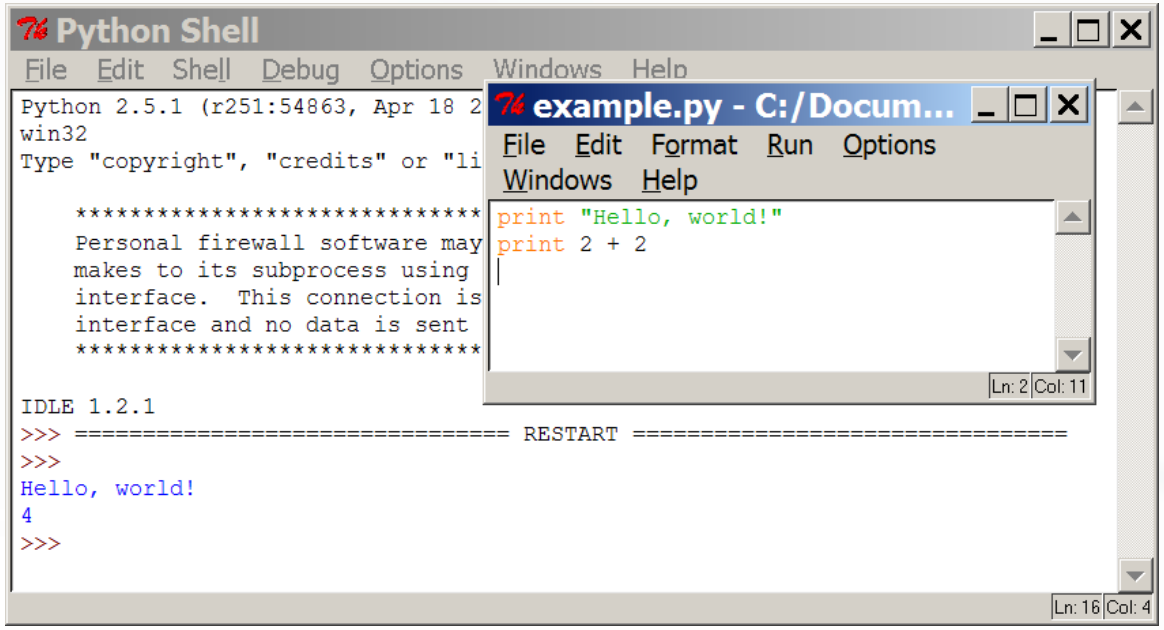
syntax: The set of legal structures and commands that can be used in a particular programming language.

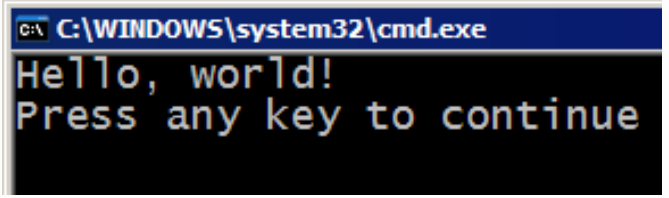
```
# Python 3: List comprehensions
>>> fruits = ['Banana', 'Apple', 'Lime']
>>> loud_fruits = [fruit.upper() for fruit
in fruits]
>>> print(loud_fruits)
['BANANA', 'APPLE', 'LIME']

# List and the enumerate function
>>> list(enumerate(fruits))
[(0, 'Banana'), (1, 'Apple'), (2, 'Lime')]
```

Programming basics

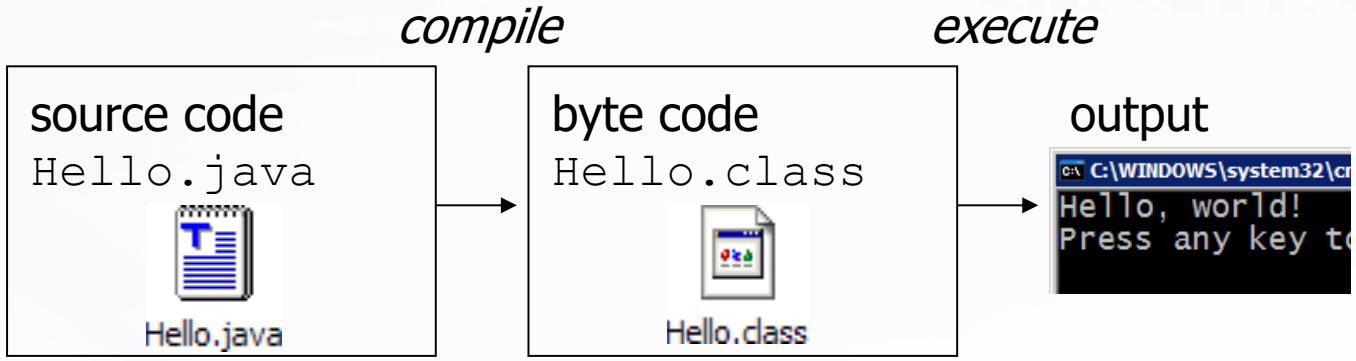
- **output**: The messages printed to the user by a program.
- **console**: The text box onto which output is printed.
  - Some source code editors pop up the console as an external window, and others contain their own console window.



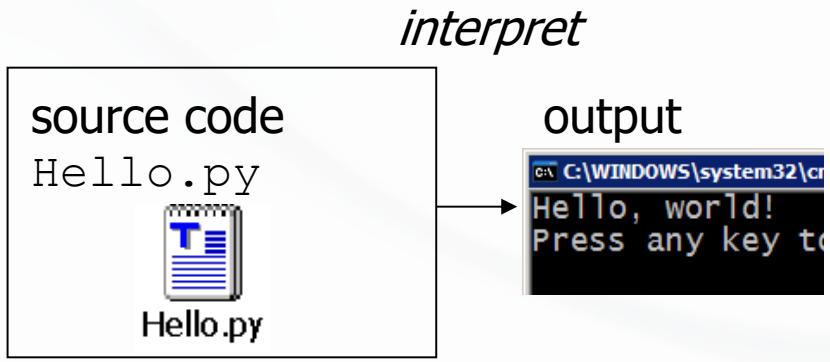


Compiling and interpreting

- Many languages require you to *compile* (translate) your program into a form that the machine understands.



- Python is instead directly *interpreted* into machine instructions.



Numeric Data Types

- The information that is stored and manipulated by computer programs is referred to as *data*.
- There are two different kinds of numbers!
  - (5, 4, 3, 6) are whole numbers – they don't have a fractional part
  - (.25, .10, .05, .01) are decimal fractions
  - Inside the computer, whole numbers and decimal fractions are represented quite differently!
  - We say that decimal fractions and whole numbers are two different *data types*.
- The data type of an object determines what values it can have and what operations can be performed on it.

Numeric Data Types

- Whole numbers are represented using the *integer* (*int* for short) data type.
- These values can be positive or negative whole numbers.
- Numbers that can have fractional parts are represented as *floating point* (or *float*) values.
- How can we tell which is which?
  - A numeric literal without a decimal point produces an int value
  - A literal that has a decimal point is represented by a float (even if the fractional part is 0)

Numeric Data Types

- Python has a special function to tell us the data type of any value.

```
>>> type(3)
<class 'int'>
>>> type(3.1)
<class 'float'>
>>> type(3.0)
<class 'float'>
>>> myInt = 32
>>> type(myInt)
<class 'int'>
```

Numeric Data Types

- Why do we need two number types?
  - Values that represent counts can't be fractional (you can't have 3 ½ quarters)
  - Most mathematical algorithms are very efficient with integers
  - The float type stores only an *approximation* to the real number being represented!
  - Since floats aren't exact, use an int whenever possible!

Expressions

- **expression**: A data value or set of operations to compute a value.

  Examples: 1 + 4 * 3
  42

- Arithmetic operators we will use:
  - + - * / addition, subtraction/negation, multiplication, division
  - % modulus, a.k.a. remainder
  - ** exponentiation

Expressions

- **precedence**: Order in which operations are computed.
  - * / % ** have a higher precedence than + -

    1 + 3 * 4 is 13

  - Parentheses can be used to force a certain order of evaluation.

    (1 + 3) * 4 is 16

Operations

Operations on ints produce ints, operations on floats produce floats (except for /).

```
>>> 3.0+4.0
7.0
>>> 3+4
7
>>> 3.0*4.0
12.0
>>> 3*4
12
```

```
>>> 10.0/3.0
3.3333333333333335
>>> 10/3
3.3333333333333335
>>> 10 // 3
3
>>> 10.0 // 3.0
3.0
```

Integer division

- Integer division produces a whole number.
- That's why 10//3 = 3!
- Think of it as 'gozinta', where 10//3 = 3 since 3 gozinta (goes into) 10 3 times (with a remainder of 1)
- 10%3 = 1 is the remainder of the integer division of 10 by 3.
- a = (a/b)(b) + (a%b)

Math commands

- Python has useful commands for performing calculations.

Command name	Description
`abs(`***value***`)`	absolute value
`ceil(`***value***`)`	rounds up
`cos(`***value***`)`	cosine, in radians
`floor(`***value***`)`	rounds down
`log(`***value***`)`	logarithm, base *e*
`log10(`***value***`)`	logarithm, base 10
`max(`***value1***`, `***value2***`)`	larger of two values
`min(`***value1***`, `***value2***`)`	smaller of two values
`round(`***value***`)`	nearest whole number
`sin(`***value***`)`	sine, in radians
`sqrt(`***value***`)`	square root

Constant	Description
`e`	2.7182818...
`pi`	3.1415926...

- To use many of these commands, you must write the following at the top of your Python program:

```
import math
```

Variables

- **variable**: A named piece of memory that can store a value.
  - Usage:
    - Compute an expression's result,
    - store that result into a variable,
    - and use that variable later in the program.

Variables

- **assignment statement**: Stores a value into a variable.
  - Syntax:

    name = ***value***

  - Examples: x = 5
    gpa = 3.14

    x 5 gpa 3.14

  - A variable that has been given a value can be used in expressions.
    x + 4 is 9

Print

- print : Produces text output on the console.
- Syntax:

  print ("***Message***")

  print (***Expression)***
  - Prints the given text message or expression value on the console, and moves the cursor down to the next line.

  print (***Item1, Item2, ..., ItemN)***
  - Prints several messages and/or expressions on the same line.

Print

- Examples:

```
print ("Hello, world!")
age = 45
print ("You have", 65 - age, "years until retirement")
```

Output:

```
Hello, world!
You have 20 years until retirement
```

Repetition (loops) and Selection (if/else)

The *for* loop

- **for loop**: Repeats a set of statements over a group of values.
  - Syntax:

    for ***variableName*** in ***groupOfValues***:
        statements

    - We indent the statements to be repeated with tabs or spaces.
    - ***variableName*** gives a name to each value, so you can refer to it in the ***statements***.
    - ***groupOfValues*** can be a range of integers, specified with the range function.

The *for* loop

- Example:

```
for x in range(1, 6):
    print (x, "squared is", x * x)
```

Output:

1 squared is 1

2 squared is 4

3 squared is 9

4 squared is 16

5 squared is 25

range

- The range function specifies a range of integers:
  - range(***start***, ***stop***) - the integers between ***start*** (inclusive) and ***stop*** (exclusive)
- It can also accept a third value specifying the change between values.
  - range(***start***, ***stop, step***) - the integers between ***start*** (inclusive) and ***stop*** (exclusive) by ***step***

range

- Example:

```
for x in range(5, 0, -1):
    print x
print ("Blastoff!”)
```

Output:

```
5
4
3
2
1
Blastoff!
```

- **Exercise:** How would we print the "99 Bottles of Beer" song?

Cumulative loops

- Some loops incrementally compute a value that is initialized outside the loop. This is sometimes called a *cumulative sum*.

```
sum = 0
for i in range(1, 11):
    sum = sum + (i * i)
print ("sum of first 10 squares is", sum)
```

Output:

```
sum of first 10 squares is 385
```

Nesting loops

- Nesting loop means putting one loop inside another

```
>>> suits = ['Spades', 'Clubs', 'Diamonds', 'Hearts']
>>> values = ['Ace', 2, 3, 4, 5, 6, 7, 8, 9, 10, 'Jack', 'Queen', 'King']
 >>> for suit in suits:
                for value in values:
                        print (str(value) + " of " + str(suit))
```

Nesting loops

- **Notice the steps in Nesting loop:**

1.It start with a suit

2.then loop through each value in the suit and printing out the card name.

3.When you've reached the end of the list of values,

4.It jump out of the nested loop and go back to the first loop to get the next suit.

5.Then you loop through all values in the second suit and print the card names.

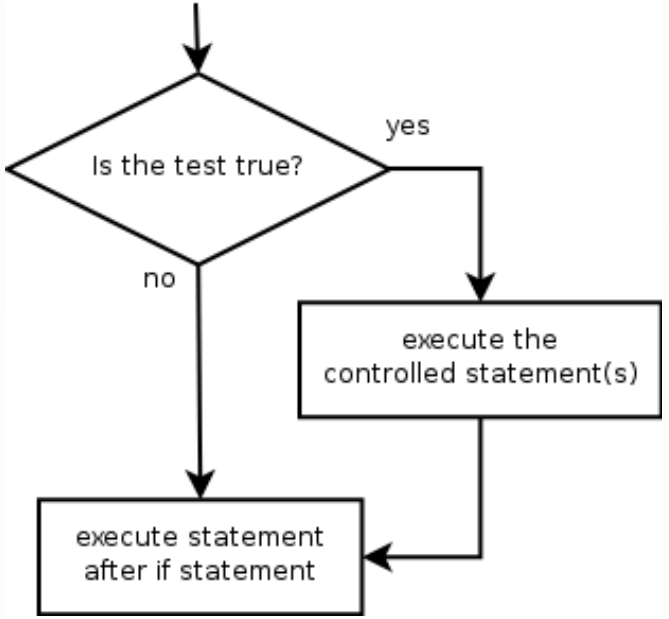
6.This process continues until all the suits and values have been looped through.

if

- **if statement**: Executes a group of statements only if a certain condition is true. Otherwise, the statements are skipped.
  - Syntax:
    if ***condition***:
        statements

- Example:

```
gpa = 3.4
if gpa > 2.0:
    print ("Your application is accepted.")
```



if/else

- **if/else statement**: Executes one block of statements if a certain condition is True, and a second block of statements if it is False.
  - Syntax:

    if ***condition***:

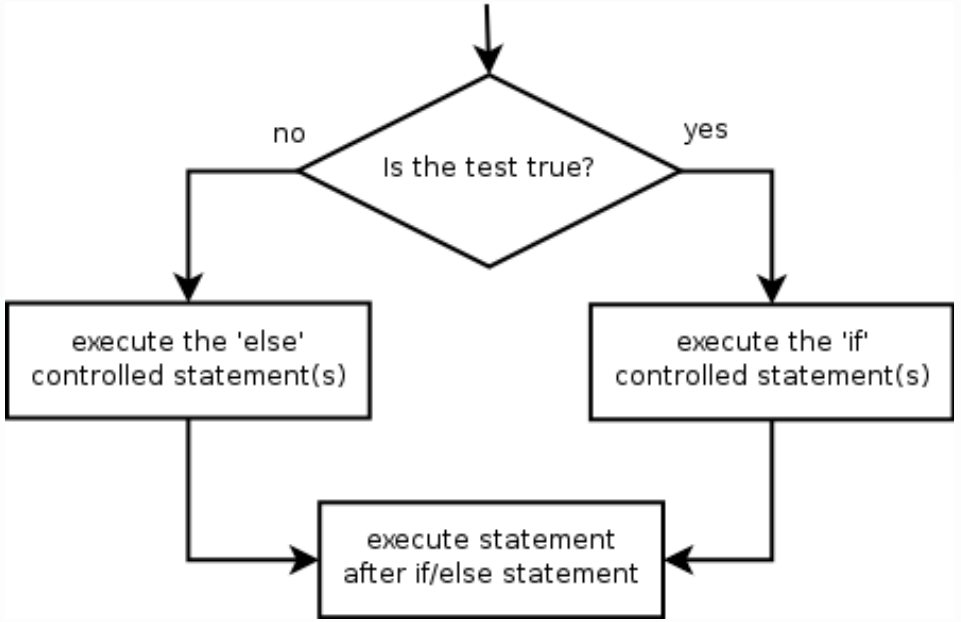
        statements

    else:

        statements

- Example:

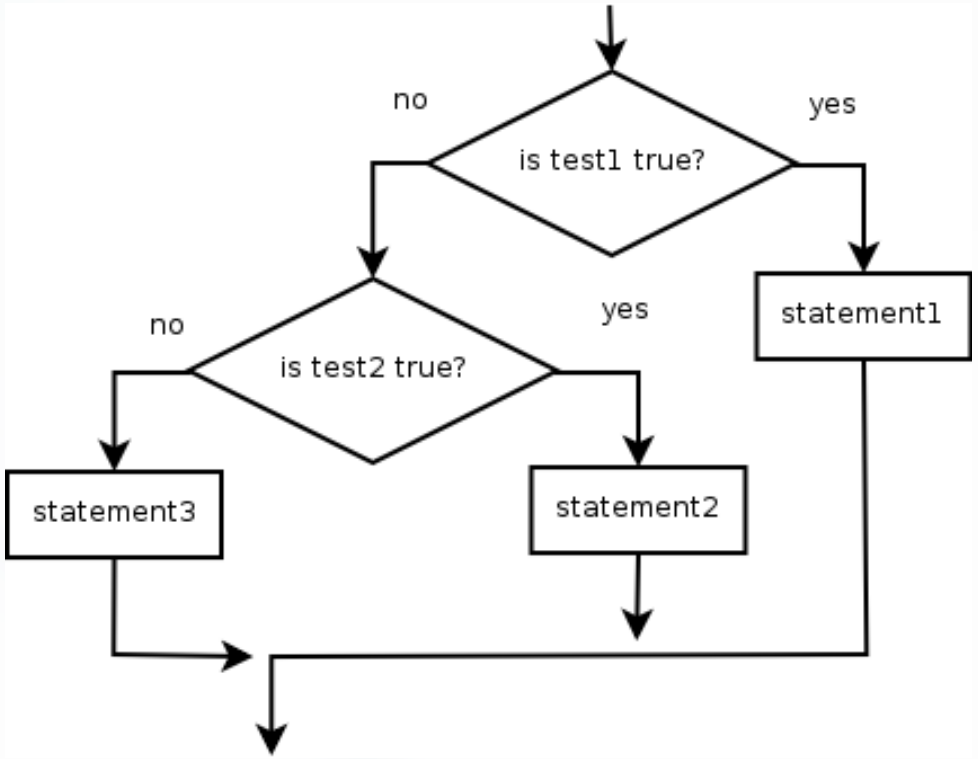
```
gpa = 1.4
if gpa > 2.0:
    print ("Welcome to Mars University!")
else:
    print ("Your application is denied.")
```



if/else

- Multiple conditions can be chained with elif ("else if"):

```
if condition:
    statements
elif condition:
    statements
else:
    statements
```

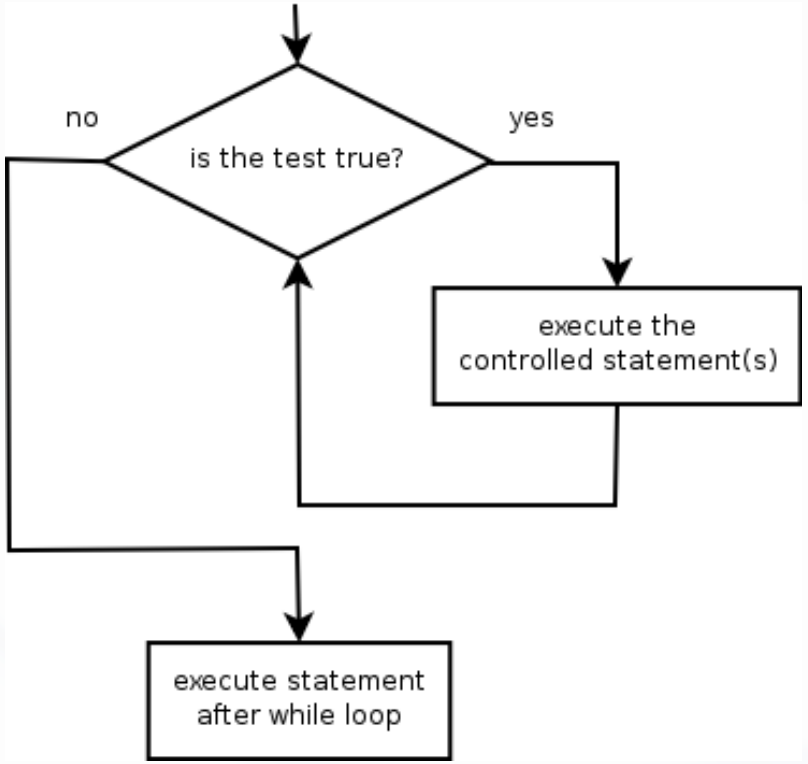


while

- **while loop**: Executes a group of statements as long as a condition is True.
  - good for *indefinite loops* (repeat an unknown number of times)

- Syntax:

```
while condition:
    statements
```



while

- Example:

```
number = 1
while number < 200:
    print (number)
    number = number * 2
```

- Output:

```
1
2
4
8
16
32
64
128
```

Logic

- Many logical expressions use *relational operators*:

Operator	Meaning	Example	Result
==	equals	`1 + 1 == 2`	`True`
!=	does not equal	`3.2 != 2.5`	`True`
<	less than	`10 < 5`	`False`
>	greater than	`10 > 5`	`True`
<=	less than or equal to	`126 <= 100`	`False`
>=	greater than or equal to	`5.0 >= 5.0`	`True`

- Logical expressions can be combined with *logical operators*:

Operator	Example	Result
`and`	`9 != 6 and 2 < 3`	`True`
`or`	`2 == 3 or -1 < 5`	`True`
`not`	`not 7 > 0`	`False`

Logic

Expression:

```
Reclass(!WELL_YIELD!)
```

Code Block:

```
def Reclass(WellYield):
    if (WellYield >= 0 and WellYield <= 10):
        return 1
    elif (WellYield > 10 and WellYield <= 20):
        return 2
    elif (WellYield > 20 and WellYield <= 30):
        return 3
    elif (WellYield > 30):
        return 4
```

Lecture 03

Working with lists

list

- You're allowed to **put multiple data types into one list**
- >>>values = ['Ace', 2, 3, 4, 5, 6, 7, 8, 9, 10, 'Jack', 'Queen', 'King']

- *Index is* an **integer** that denotes each **item's place** in the list.
- >>>print (values[0])
- ‘Ace’

- The list **starts with index 0** and for each item in the list, the index **increments by one**

Change the order of the items

- **Sort a list as Alphabetical order**
- >>> suits = ['Spades', 'Clubs', 'Diamonds', 'Hearts']
- >>> suits.sort()
- >>> print (suits)
- 'Clubs', 'Diamonds', 'Hearts', 'Spades'

- **Sort a list in reverse alphabetical order:**
- >>> suits.reverse()
- >>> print (suits)
- 'Spades', 'Hearts', 'Diamonds', 'Clubs'

Inserting items and combining lists

- >>> listOne = [101,102,103]
- >>> listTwo = [104,105,106]
- >>> listThree = listOne + listTwo
- >>> print (listThree)
- [101, 102, 103, 104, 105, 106]
- **Python treats the addition as appending listTwo to listOne.**

  >>> listThree += [107]
- >>> print (listThree)
- [101, 102, 103, 104, 105, 106, 107]
- >>> listThree.append(108)
- >>> print (listThree)
- [101, 102, 103, 104, 105, 106, 107, 108]

Inserting items and combining lists

- **The append() method could also be used to put an item at the end of the list**
- >>> listThree.insert(4, 999)
- >>> print (listThree)
- [101, 102, 103, 104, 999, 105, 106, 107, 108]

Getting the length of a list

- >>> myList = [4,9,12,3,56,133,27,3]
- >>> print (len(myList))
- 8
- **len()** gives you the exact number of items in the list
- To get the *index* of the final item, you would need to use **len(myList) - 1**.

Lecture 03

Strings

Strings

- **string**: A sequence of text characters in a program.
  - Strings start and end with quotation mark " or apostrophe ' characters.
  - Examples:

    "hello"
    "This is a string"
    "This, too, is a string. It can be very long!"

- A string may span across multiple lines or contain a " character.

"This is not
a legal String."

"This is not a "legal" String either."

Indexes

- Characters in a string are numbered with *indexes* starting at 0:
  - Example:
    name = "P. Diddy"

index	0	1	2	3	4	5	6	7
character	P	.		D	i	d	d	y

Indexes

- Accessing an individual character of a string:

  variableName [***index***]

  - Example:

    print (name, "starts with", **name[0])**

    Output:

    P. Diddy starts with P

String properties

- len(***string***) - number of characters in a string (including spaces)
- str.lower(***string***) - lowercase version of a string
- str.upper(***string***) - uppercase version of a string

String properties

- Example:

  name = "Martin Douglas Stepp"

  length = **len(name)**

  big_name = **str.upper(name)**

  print (big_name, "has", length, "characters")

  Output:

  MARTIN DOUGLAS STEPP has 20 characters

Text processing

- **text processing**: Examining, editing, formatting text.
  - often uses loops that examine the characters of a string one by one

- A for loop can examine each character in a string in sequence.
  - Example:

```
for c in "booyah":
    print c
```

Output:

```
b
o
o
y
a
h
```

Reference

https://docs.python.org/3/tutorial/